

Trabalho Prático 1: Alinhamento Estratégico de Carteiras

Documentação Técnica (README.pdf) | Problema: Núcleo de Consenso Ordenado (LCS) | Linguagem: Java 17+

1. Descrição e Modelagem da Solução Desenvolvida

O objetivo deste trabalho prático é determinar o **Núcleo de Consenso Ordenado** entre duas carteiras recomendadas de ações da B3, emitidas por duas corretoras distintas (**Corretora A** e **Corretora B**). A meta é identificar a maior sequência de ações em comum que preserva a ordem relativa de recomendação em ambas as casas.

1.1. Modelagem do Problema (LCS)

Este problema é mapeado para o problema clássico da **Maior Subsequência Comum** (*Longest Common Subsequence* — LCS):

- Seja **A** a lista de **M** ações recomendadas pela Corretora A.
- Seja **B** a lista de **N** ações recomendadas pela Corretora B.
- Deseja-se encontrar uma subsequência comum **C** de comprimento máximo **K** que esteja presente em **A** e em **B** respeitando a ordem de prioridade.

1.2. Abordagem por Programação Dinâmica

A solução utiliza **Programação Dinâmica** para resolver o problema de forma eficiente construindo as respostas de subproblemas menores em uma tabela de memória.

Definição da Tabela DP:

$dp[i][j]$ armazena o tamanho da maior subsequência comum entre os i primeiros ativos da Corretora A e os j primeiros ativos da Corretora B.

1.3. Relação de Recorrência

A regra de cálculo para preencher a matriz `dp` é definida por:

$$dp[i][j] = \begin{cases} 0 & \text{se } i = 0 \text{ ou } j = 0 \\ dp[i-1][j-1] + 1 & \text{se } a_{i-1} == b_{j-1} \\ \max(dp[i-1][j], dp[i][j-1]) & \text{se } a_{i-1} \neq b_{j-1} \end{cases}$$

1.4. Preenchimento e Reconstrução do Resultado

O algoritmo aloca a matriz `dp` de dimensão $(M+1) \times (N+1)$ e preenche seus valores de forma iterativa (bottom-up).

Após computar o valor máximo $K = dp[M][N]$, realiza-se o *backtracking* partindo da posição (M, N) até $(0, 0)$ para recuperar os nomes dos ativos alinhados. A lista resultante é invertida para manter a ordem original de recomendação.

2. Análise de Complexidade de Tempo

A análise de tempo descreve a eficiência do algoritmo em função dos tamanhos **M** (Corretora A) e **N** (Corretora B).

2.1. Detalhamento por Etapa

- **Leitura da Entrada:** Leitura das ações da Corretora A e da Corretora B em tempo $O(M + N)$.
- **Preenchimento da Matriz DP:** Consiste em dois laços encadeados percorrendo $M \times N$ células. Cada célula compara duas strings de ticker com custo constante $O(1)$. O tempo desta etapa é $\Theta(M \times N)$.
- **Reconstrução por Backtracking:** Rastreamento regressivo que realiza no máximo $M + N$ passos em tempo $O(M + N)$.
- **Impressão da Saída:** Exibição dos K ativos alinhados em tempo $O(K)$, onde $K \leq \min(M, N)$.

Complexidade de Tempo Total:

$$T(M, N) = \Theta(M \times N)$$

O tempo de execução do algoritmo é proporcional ao produto dos tamanhos das duas listas, garantindo resposta quase instantânea para a faixa de entradas informada ($1 \leq M, N \leq 1000$).

2.2. Resumo da Complexidade de Tempo

Etapa da Solução		Complexidade de Tempo	
Leitura dos Dados (STDIN)		$O(M + N)$	
Preenchimento da Matriz DP		$\Theta(M \times N)$	
Backtracking (Recuperação da Sequência)		$O(M + N)$	
Inversão e Impressão do Consenso		$O(K)$	
COMPLEXIDADE DE TEMPO TOTAL		$\Theta(M \times N)$	

3. Descrição do Uso de IA (Inteligência Artificial)

Descreve-se a seguir como a inteligência artificial (Google Antigravity / Gemini) foi utilizada durante o desenvolvimento do trabalho:

1. **Auxílio na Estruturação do Código:**
A IA auxiliou no desenvolvimento em Java separando a lógica da solução em `LCS.java` e a interface de leitura/escrita em `Main.java`.
2. **Criação de Testes de Validação:**
A IA foi utilizada para criar scripts automatizados de testes cobrindo exemplos padrão e casos de borda (listas disjuntas, totalmente alinhadas e com ordens alternadas).
3. **Elaboração da Documentação:**
A IA auxiliou na redação clara da modelagem do problema, organização da análise de complexidade de tempo e compilação do arquivo `README.pdf`.